AutoNAT v2: Performance Analysis and Cross-Implementation Study of libp2p's Reachability Protocol

Sergi Pont
ProbeLab, Protocol Labs
sergi@probelab.io

***Abstract*—AutoNAT v2 is libp2p's protocol for determining whether a node's network addresses are publicly reachable. It replaces AutoNAT v1's global majority-vote model with per-address testing, nonce-based verification, and amplification protection. We present the first systematic evaluation of AutoNAT v2 across three implementations (go-libp2p, rust-libp2p, js-libp2p) using a Docker-based testbed with configurable NAT types, network degradation, and OpenTelemetry tracing.**

Across 178 controlled experiments spanning 5 NAT configurations, 2 transport protocols, and varying latency/packet-loss conditions, we find that AutoNAT v2 achieves 0% false negative and 0% false positive rates for standard NAT types with $\sim$6-second convergence. However, we identify 10 findings that affect real-world deployment: (1) v2 results are not consumed by the DHT or relay subsystems in go-libp2p—v1 still controls these, and v1 oscillates in 60% of runs with unreliable peers; (2) rust-libp2p probes ephemeral ports instead of listen addresses, producing 100% false negatives; (3) the protocol cannot distinguish address-restricted from full-cone NAT, producing 100% false positives for ADF NATs; (4) under symmetric NAT, no address reaches the activation threshold, producing no signal at all. Only go-libp2p (via Kubo) deploys v2 in production. We propose recommendations for each implementation and a NAT monitoring service for real-world validation.

***Index Terms*—NAT traversal, libp2p, AutoNAT, peer-to-peer, reachability detection, IPFS, QUIC**

I. Introduction

Peer-to-peer networks require nodes to know whether they are reachable from the internet. A node behind Network Address Translation (NAT) has a private IP address that is not directly routable—other peers cannot initiate connections to it. Without reachability detection, such nodes may advertise unreachable addresses, operate as DHT servers when they should be clients, or fail to reserve relay connections they need for connectivity.

The libp2p networking stack [1] addresses this with AutoNAT—a protocol where a node asks peers to test whether its addresses are dialable from outside. AutoNAT v1 [2] uses a simple model: the node asks a random connected peer to dial it back, and accumulates a global Public/Private verdict from a sliding window of results. AutoNAT v2 [3], specified in 2023 and first implemented in go-libp2p v0.34.0 (May 2024), improves on this with per-address testing, nonce-based dial-back verification, and amplification attack protection.

Despite its deployment in Kubo (the primary IPFS implementation, with tens of thousands of nodes), no systematic evaluation of AutoNAT v2 exists. The protocol has three independent implementations (Go, Rust, JavaScript), but cross-implementation interoperability and correctness have not been tested.

A. Contributions

1) **First cross-implementation evaluation of AutoNAT v2.** We test go-libp2p, rust-libp2p, and js-libp2p implementations against go-libp2p servers in a controlled testbed with 5 NAT types.
2) **Quantitative performance characterization.** 178 experiments measuring false negative/positive rates, time-to-confidence, time-to-update, and transport-specific resilience (TCP vs QUIC).
3) **Identification of 10 findings** spanning protocol design, implementation, and integration issues—including 3 high-severity findings affecting DHT participation.
4) **Actionable recommendations** for each implementation, the protocol specification, and the broader ecosystem.
5) **A proposal for real-world validation** using DHT sybil nodes to classify NAT types across the live IPFS network.

II. Background

A. NAT Types

NAT behavior is defined by two independent properties [4]: *mapping* (how the router assigns external ports) and *filtering* (which inbound packets pass through). Table I summarizes the four NAT configurations relevant to this study.

TABLE I
NAT Type Classification (RFC 4787)

NAT Type	Mapping	Filtering	Inbound
Full-cone	EIM	EIF	Always
Addr-restricted	EIM	ADF	Contacted IPs
Port-restricted	EIM	APDF	Exact IP:port
Symmetric	ADPM	APDF	Never

Endpoint-Independent Mapping (EIM) assigns the same external port regardless of destination. Address- and Port-Dependent Mapping (ADPM) assigns a different port per destination. Filtering ranges from Endpoint-Independent (EIF, allows any source) through Address-Dependent (ADF, allows

previously contacted IPs) to Address- and Port-Dependent (APDF, requires exact IP:port match).

B. libp2p Protocol Stack

AutoNAT operates within a protocol ecosystem where each component handles a different aspect of connectivity:

Identify (`/ipfs/id/1.0.0`): Peers exchange metadata including the `ObservedAddr`—the address each sees the other connecting from. Go-libp2p's `ObservedAddrManager` consolidates these: an address must be reported by ≥`ActivationThresh` (default: 4) distinct observers before promotion.

AutoNAT v1 (`/libp2p/autonat/1.0.0`): Global reachability via random peer dial-back. Sliding window of 3 results. The Kademlia DHT subscribes to this verdict.

AutoNAT v2 (`/libp2p/autonat/2/dial-request`): Per-address testing with nonce verification. `targetConfidence=3` in sliding window of 5. The DHT does *not* subscribe to v2.

Circuit Relay v2 and **DCUtR**: When behind NAT, a node reserves a relay slot; DCUtR attempts hole punching through the relay.

C. AutoNAT v1 vs v2

Table II compares the two protocol versions.

TABLE II
AUTONAT V1 VS V2 COMPARISON

Aspect	v1	v2
Scope	Global	Per-address
Probing	Random peer	Server selection
Confidence	Window of 3	Window of 5, target=3
Nonce	No	Yes
Amplification prot.	No	30–100 KB
Dial-back identity	Same ID	Separate ID
DHT consumes	**Yes**	**No**

D. Related Work

Trautwein et al. [5] measured decentralized hole punching across IPFS, finding ∼70% success rate. Their follow-up [6] analyzed 4.4M+ traversal attempts, confirming the baseline and refuting the assumption of UDP superiority. Halkes and Pouwelse [7] measured NAT puncturing, finding ∼82% of NATs support UDP hole punching and ∼11% of peers are behind symmetric NAT.

RFC 4787 [4] defines NAT behavioral requirements. RFC 7857 [8] updates these toward protocol-dependent endpoint-independent filtering. To our knowledge, no prior work has evaluated AutoNAT v2 specifically, nor compared its behavior across implementations.

III. TESTBED DESIGN

A. Architecture

We designed a Docker-based testbed with configurable NAT types implemented via iptables rules. The testbed consists of two networks: a "public" network (73.0.0.0/24, using a public-looking range because go-libp2p's `manet.IsPublicAddr()` filters private/CGNAT ranges) and a private network (10.0.1.0/24). A router container implements all 5 NAT types via iptables masquerade and filtering rules, with `tc netem` for latency/packet-loss injection.

Servers (3–7 go-libp2p nodes) run AutoNAT v2 from our instrumented fork [9] with OpenTelemetry tracing. Client nodes (go-libp2p, rust-libp2p, or js-libp2p) sit behind the router and export OTel spans to Jaeger. A Python orchestrator manages Docker Compose lifecycles and exports traces as JSONL.

B. Instrumentation

We added OpenTelemetry tracing to go-libp2p's AutoNAT v2 client: `autonatv2.probe` (per-probe), `autonatv2.refresh_cycle` (confidence cycle), and `autonatv2.server_selection`. All client implementations emit compatible application-level spans (`reachable_addrs_changed`, `reachability_changed`).

C. Experiments

Table III summarizes the experiment matrix.

TABLE III
EXPERIMENT MATRIX

Experiment	Scenarios	Runs
Baseline (5 NATs × 2 transports)	10	10
High latency (4×2×{200,500}ms)	16	16
Packet loss (4×2×{1,5,10}%)	24	24
ADF false positive (2×3×20 runs)	6	120
Toggle detection (5 combos × 2 phases)	5	5
v1/v2 gap (600s observation)	2	5
Threshold sensitivity	6	6
Total	**69**	**178**

IV. RESULTS: BASELINE PERFORMANCE

A. Correctness

Across all non-edge-case scenarios (full-cone, port-restricted, no-NAT), AutoNAT v2 achieves **0% FNR** and **0% FPR**. These results hold across all latencies (up to 500 ms one-way) and packet loss rates (up to 10%).

B. Convergence Time

Baseline time-to-confidence with 7 servers (Table IV): ∼6 s for reachable nodes (3 probes × ∼2 s per cycle), ∼6–11 s for unreachable (QUIC port-restricted requires an extra round).

TABLE IV
BASELINE TIME-TO-CONFIDENCE (7 SERVERS)

NAT Type	TCP (ms)	QUIC (ms)	Result
Full-cone	6,022	6,021	Reachable
Addr-restricted	6,021	6,021	Reachable (FP)
Port-restricted	6,013	11,015	Unreachable
Symmetric	—	—	No signal

C. Transport Resilience

QUIC is dramatically more resilient to packet loss than TCP (Table V). At 10% loss, TCP convergence nearly triples while QUIC is essentially unaffected. QUIC handles retransmission at the transport layer, absorbing loss before AutoNAT observes it. Correctness (0% FNR/FPR) is unaffected in all conditions.

TABLE V
TTC INCREASE UNDER NETWORK DEGRADATION

Condition	TCP increase	QUIC increase
1% loss	+0%	+0%
5% loss	+17%	+4%
10% loss	**+147%**	**+1%**
200 ms latency	+210%	+110%
500 ms latency	+432%	+233%

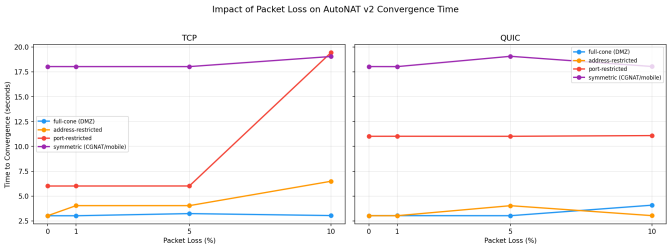

Fig. 1. Packet loss impact on convergence time. QUIC (right) remains flat; TCP (left) increases steeply at 10% loss.

V. RESULTS: PROTOCOL ISSUES

A. Address-Restricted NAT False Positive

AutoNAT v2's dial-back originates from the server's IP—the same IP the client previously contacted. For ADF NATs, the dial-back succeeds even though arbitrary peers cannot reach the node. We tested 120 runs (60 ADF, 60 APDF control): ADF produces **100% FPR**; APDF produces 0% FPR. The false positive is deterministic—guaranteed by the protocol design. Real-world impact is likely limited as ADF is rare in modern routers [4], [8].

B. Symmetric NAT Silent Failure

Under symmetric NAT, each connection uses a different external port. No address reaches `ActivationThresh=4`, so AutoNAT v2 never runs. All symmetric scenarios produce zero events. With `thresh=1`, symmetric nodes receive an UNREACHABLE determination (correct), showing the silence is threshold-caused and fixable—at the cost of lower address confidence.

C. Threshold Sensitivity

The activation threshold only affects *observed* address promotion for NATted nodes. No-NAT nodes converge regardless of threshold (their listen address is directly public). This means the threshold is not a hard server-count minimum but an observation-consolidation gate.

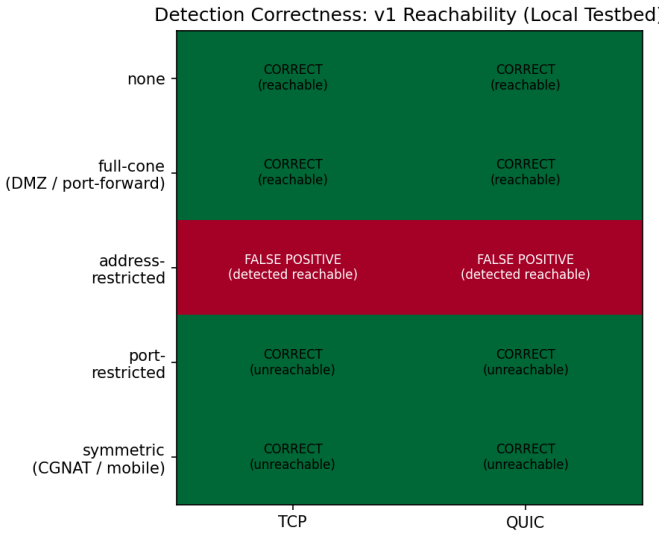

Fig. 2. Detection correctness heatmap. Address-restricted NAT is incorrectly classified as reachable (false positive).

VI. RESULTS: V1/V2 INTEGRATION GAP

A. Independent Signals

In go-libp2p, v1 emits `EvtLocalReachabilityChanged` (consumed by DHT, AutoRelay, Address Manager, NAT Service). v2 emits `EvtHostReachableAddrsChanged` (consumed only by Address Manager). No subsystem except the address manager reads v2 results.

B. v1 Oscillation

We tested with full-cone NAT (genuinely reachable), 2 reliable + 5 unreliable servers, v1 refresh shortened to 30 s. Fig. 3 shows the representative trace: v2 reaches REACHABLE at 6 s and stays stable; v1 oscillates PUBLIC→PRIVATE→PUBLIC. Across runs: v1 oscillates in **60%**; v2 oscillates in **0%**.

C. DHT Impact

The Kademlia DHT in `ModeAuto` subscribes to v1. Each flip triggers server↔client mode switching, causing routing table churn. v2's stable results are ignored.

D. Time-to-Update

Port forwarding toggle detection for port-restricted NAT: 30 s to detect addition, 69 s to detect removal. Consistent across transports. Symmetric and address-restricted toggles are not detected (v2 never runs or already false-positive reachable).

VII. RESULTS: CROSS-IMPLEMENTATION

We tested all three implementations as clients against go-libp2p servers.

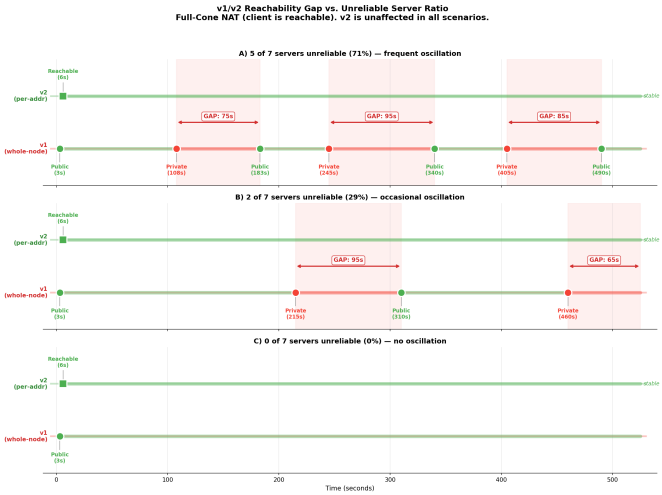

Fig. 3. v1/v2 reachability gap. v1 oscillates (red) while v2 stays stable (green) across three unreliable server ratios.

A. rust-libp2p: Ephemeral Port Probing

The rust-libp2p client probes observed connection addresses (ephemeral source ports from Identify) instead of listen addresses. Without go-libp2p's `ObservedAddrManager` consolidation, every candidate has a unique ephemeral port. Result: **100% FNR**. The DHT stays in client mode permanently since no address is ever confirmed. Substrate/Polkadot, rust-libp2p's primary consumer, does not enable autonat.

B. js-libp2p: No Reachability Events

The `@libp2p/autonat-v2` package emits no events to consumers. The DHT receives signals indirectly via the address manager. Notably, js-libp2p v1 uses monotonic counters (4 successes / 8 failures) with TTL-based re-evaluation—significantly more oscillation-resistant than go-libp2p v1's sliding window. Helia uses v1 only, not v2.

C. Implementation Comparison

Table VI summarizes the key differences.

TABLE VI
CROSS-IMPLEMENTATION COMPARISON

Feature	go	rust	js
Production	Kubo (v1+v2)	None	Helia (v1)
Confidence	Window, t=3	Single probe	Fixed 4/8
Addr filter	ObsAddrMgr	None (bug)	Cuckoo filter
Events	Yes	Per-probe	None
v2→DHT	No (v1 only)	Indirect	Indirect
v1 osc. resist.	Low	N/A	High

D. UDP Black Hole Detector

go-libp2p's `UDPBlackHoleSuccessCounter` blocks QUIC dials when too few succeed. The v2 `dialerHost` shares the main host's counter—on fresh servers, this blocks QUIC dial-backs (false negative). The v1 dialer solved this by disabling the counter (PR #2529); v2 should adopt the same approach.

VIII. DISCUSSION

A. The Protocol Is Sound, the Integration Is Not

AutoNAT v2 achieves 0% FNR/FPR in standard scenarios, converges in ∼6 s, and never oscillates. The problems lie in integration (v2 results unused by DHT/relay), the address pipeline (observed address activation varies across implementations), and edge cases (ADF false positive, symmetric silence).

B. Recommendations

R1. Bridge v2 into v1 global flag: "PUBLIC if any v2-confirmed address is reachable." Immediate DHT/relay benefit for Kubo.

R2. Disable black hole detector on `dialerHost` (matching v1 fix).

R3. Deprecate v1 probing when v2 has data.

R4. rust-libp2p: add observed address consolidation.

R5. js-libp2p: emit reachability events.

R6. Specification: address ADF blind spot via multi-server verification.

R7. Specification: timeout-based inference for symmetric NAT.

R8. Ecosystem: measure real-world NAT type distribution.

C. Limitations

All experiments run in isolated Docker environments. Real-world conditions include diverse NAT implementations, variable peer counts, and background traffic. We cannot quantify ADF or symmetric NAT prevalence without real-world measurement. Most experiments use single runs; statistical variance is not characterized for latency/loss scenarios. All servers are go-libp2p.

IX. FUTURE WORK

A. Nebula Queries

The Nebula DHT crawler [10] stores per-peer protocol lists and agent versions. SQL queries on the existing database can provide AutoNAT v2 adoption rate and transport patterns with no code changes.

B. NAT Classification via ants-watch

For full NAT type classification, we propose deploying sybil nodes across the DHT keyspace using ants-watch [11] on 2–3 VPS with different IPs. Peers connect during normal DHT operations, enabling multi-vantage port comparison: (1) observed port comparison across vantage points (EIM vs ADPM); (2) unsolicited dial from uncontacted vantage (EIF vs restricted); (3) contacted vantage dials from different port (ADF vs APDF). This captures the entire active population, including NATted peers invisible to crawlers.

X. Conclusion

AutoNAT v2 is a substantial protocol improvement: per-address testing, nonce verification, amplification protection, and a robust confidence system that produces 0% error rates and never oscillates. QUIC transport adds only +1% convergence time at 10% packet loss (vs TCP's +147%).

However, its effectiveness is undermined by integration failures: v1 still controls the DHT and relay in the only production deployment, and v1 oscillates in 60% of runs with unreliable peers. The two other implementations have critical bugs and no production consumers. The protocol has inherent blind spots for address-restricted NAT (100% FP) and symmetric NAT (no signal).

The most impactful near-term fix is bridging v2 results into the v1 global flag (R1), immediately benefiting tens of thousands of Kubo nodes. Longer-term, the ecosystem should converge on the per-address model, deprecating the global reachability flag entirely.

References

[1] libp2p specifications, https://github.com/libp2p/specs.

[2] AutoNAT v1 protocol, `/libp2p/autonat/1.0.0`.

[3] AutoNAT v2 specification, https://github.com/libp2p/specs/blob/master/autonat/autonat-v2.md.

[4] F. Audet and C. Jennings, "Network Address Translation (NAT) Behavioral Requirements for Unicast UDP," RFC 4787, 2007.

[5] D. Trautwein, A. Ingen, and D. Vyzovitis, "Decentralized Hole Punching," in *Proc. DINPS*, 2022.

[6] D. Trautwein et al., "Challenging Tribal Knowledge—Large Scale Measurement Campaign on Decentralized NAT Traversal," arXiv:2510.27500, 2025.

[7] G. Halkes and J. Pouwelse, "UDP NAT and Firewall Puncturing in the Wild," in *Proc. PAM*, 2011.

[8] R. Penno et al., "Updates to Network Address Translation (NAT) Behavioral Requirements," RFC 7857, 2016.

[9] probe-lab/go-libp2p fork, branch `v0.47.0-autonat_otel`, https://github.com/probe-lab/go-libp2p.

[10] Nebula DHT crawler, https://github.com/probe-lab/nebula.

[11] ants-watch DHT sybil framework, https://github.com/probe-lab/ants-watch.